

01 / UNDERSTAND THE ASSIGNMENT

What am I asking the model to do?

You know the analysis. Make the required behavior explicit, then inspect the evidence.

THE WORKING PROMISE

We are reducing unnecessary choices and building reproducible checks. We are not promising identical model responses.

Teach a collaboration workflow, not a collection of magic prompts. Ask for one kind of work at a time, and review the result before moving to the next stage.

Stage	Ask for	Review before moving on
Understand	Explain inputs, transformations, outputs, and dependencies.	Does the explanation match the source? What is inferred?
Specify	State the behavior, environment, scope, and expected results.	Which choices must the analyst decide?
Translate	Implement one agreed unit in Python.	Does the code address the requirements without unrelated changes?
Check	Execute reviewed code against established expectations.	What passed, what failed, and what remains unchecked?
Improve	Propose a separate refactor or optimization.	Do required behaviors still hold after the change?

Start with explanation, not a rewrite

Explain this SAS step without translating it. Identify its inputs, output columns, transformations, and dependencies. Separate statements supported by the code from assumptions. Flag missing definitions before describing affected behavior.

How: Check the explanation against the source and relevant documentation. Treat comments as context to verify, not automatic proof of behavior.

Why: An implementation can look reasonable while solving a different problem. Agreeing on the task makes that mismatch easier to notice.

Why not: Do not ask for translation, a new architecture, and a different analytical method in one request. You would lose a clear basis for comparison.

KEEP THE UNIT COHERENT

Chunk at an understandable input/output boundary, not an arbitrary line count. Carry needed formats, macro definitions, grouping and ordering assumptions with the chunk. Ask for missing context rather than inventing it.

Use only approved source and synthetic or otherwise authorized data. A model response is a proposal, not permission to run code or access additional data.

02 / SPECIFY THE REQUEST

What does the model need to know?

Use practical headings. Make consequential choices explicit, not every possible choice.

Completed example: row-wise MEAN

Heading	A bounded request you can adapt
Task	Translate only the row-wise SAS MEAN calculation into a Python function named <code>candidate(df)</code> .
Source and context	Use the attached SAS source. The Python input contains <code>case_id</code> , <code>row_id</code> , <code>x</code> , <code>y</code> , <code>z</code> . Only <code>x</code> , <code>y</code> , <code>z</code> are measurements; the other columns are labels.
Environment	Use Python with pandas in the approved environment. The measurement columns are float64 and use NaN for ordinary numeric missing values. Record actual versions separately.
Required behavior	Average <code>x</code> , <code>y</code> , <code>z</code> within each row, excluding missing values. Keep an all-missing average missing. Preserve zeros and negatives. Return a new table without changing the input; retain its rows, order, index and columns, and append numeric average.
Deliverable	Return code and a brief mapping to the requirements. Do not install packages, read unrelated files, create files, or execute code in this step.
Uncertainty and checks	Flag missing information or conflicts before implementing affected behavior. Do not silently invent a rule. State which checks are proposed, not executed. Use the supplied expected values; do not alter them to match the implementation.

The source to attach or paste after the request

```
data test;
  input x y z;
  average = mean(x, y, z);
  datalines;
1 2 3
4 . 6
. . .
;
run;
```

Expected averages: 2.0, 5.0, and missing. The notebook adds input labels and preservation requirements for testing; these are explicit interface requirements, not hidden SAS behavior. [SAS; Project]

How: Supply the actual source, schema, behavior rules, and approved tools. Include only the context needed for the bounded task.

Why: Stated choices can be inspected. Unstated choices must be found before they can be accepted.

Why not: More text is not automatically more useful. Remove conflicting instructions and unrelated conversation history. A concise prompt that passes the checks is a valid outcome.

Sources: [SAS](#) | [Project](#)

03 / CHECK THE EVIDENCE

How do I check the result?

Define expected behavior before judging the candidate. Keep the expected values separate from its calculation.

THREE DIFFERENT CLAIMS

The model displayed an answer. The code executed. The observed results met specified checks. Record only the claim supported by the evidence you actually have.

MEAN: make the checks visible

Input x, y, z	Expected average	What this distinguishes
1, 2, 3	2.0	Ordinary arithmetic.
4, missing, 6	5.0	Exclude missing values; do not use a fixed denominator of three.
missing, missing, missing	Missing	Do not turn an all-missing row into zero.
0, missing, missing	0.0	Zero is an observed value.
missing, -6, missing	-6.0	Preserve a lone negative measurement.

Expected values above follow the stated MEAN rule and the shared repository fixture. They are not captured SAS output. [SAS; Fixture]

Three named acceptance tests

T1. Known answers: Check all three original rows, including their missing positions.

T2. Discriminating cases: Check the added zero and negative cases.

T3. Preservation: Check that the input is unchanged and that a new output preserves labels, values, row order, index and types while adding average. [Project]

For this fixture, pandas computes across columns with axis=1 and excludes missing values with skipna=True. Check missing results with pandas.isna(), not equality to NaN. [pandas; Missing]

How: Inspect the code, execute it in the approved environment, and retain the actual output. Review implementation requirements as well as values. For other calculations, agree numerical tolerances before comparing.

Why: A check is useful when it can reject a relevant mistake. Include cases that distinguish plausible interpretations, not only easy inputs.

Why not: Do not let the candidate implementation generate its own expected answers. Model-suggested tests need independent review; a second model is not an execution record.

ACCEPTANCE HAS A BOUNDARY

Three tests are a teaching structure, not proof of general equivalence. Record the supported scope and unresolved limits. A teaching pass is not production approval.

Sources: [SAS](#) | [Fixture](#) | [Project](#) | [pandas](#) | [Missing](#)

04 / REVISE OR PAUSE

What do I do when something is wrong?

Use a mismatch as evidence. Request the smallest justified correction.

A BOUNDED REPAIR REQUEST

For input [4, missing, 6], the required average is 5.0, but this run returned 3.3333333333. The attached code filled missing values with zero. Explain the mismatch, then revise only the calculation. Preserve the interface and existing checks. Return code without executing it. Do not change expected values.

How: Preserve the original response and failed output. Include the source/input version, expected result, actual result and relevant code or traceback. Rerun every acceptance test after correction.

Why: A narrow change is easier to review than a replacement program. Rerunning the earlier checks helps reveal regressions.

Why not: Do not keep saying “try again” without evidence, or change expected values to make a failure disappear. A genuine correction to a requirement must be reviewed and versioned separately.

Pause when a consequential choice is unresolved

Missing macro or format definitions; conflicting source behavior and intended behavior; unknown join or ordering rules; an unapproved dependency; or changes to the analytical method. Identify the question and its owner. Do not guess to keep the conversion moving.

Keep the conversation, workspace, and runtime distinct

For controlled A/B trials, keep the source, starting instructions, model, and visible settings fixed. Start fresh conversations, avoid carrying prior outputs forward, and isolate workspaces when tools can read files. This does not guarantee identical responses. Bedrock parameters differ by model; record unavailable settings as unknown. [AWS]

In a notebook, inspect the cells, restart the kernel, then run from the beginning. Kernel restart clears in-memory state, not files on disk. Review saved outputs before sharing: notebook documents can retain them. [Jupyter]

Close each exercise with an evidence record

Required behavior	Evidence	Limits	Decision
What had to remain true?	What ran, in which environment, and what happened?	What was not checked or remains unresolved?	Accept for the stated teaching scope, revise, or seek clarification.

Keep with the record: model identifier, access application, visible settings, source/prompt/code versions, run output, and reviewer decision. Do not paste credentials into the record.

Sources: [AWS](#) | [Jupyter](#)